

Program Documentation - Nand2Tetris (v7 - Jack Language)

Student Name: Adnan Mohammed Shelim and Alex Brijo Sebastian

Program Name: Reaction Duel

Date: February 1st 2026

1. What does the program do?

1.1. What is the main purpose of the program?

The program is a game inspired by gunslinger duels done in the wild west. In this game two players wait for a visual signal and then whoever presses their button first wins. The visual que is randomised.

1.2. What input does it take and what output does it return?

Input:

- Key presses from the Keyboard.
- Player A uses the 'A' key (ASCII 65).
- Player B uses the 'B' key (ASCII 66)
- The program checks which of these keys are clicked first.

Output:

- Text prompts on the screen ("Wait for it...", "DRAW!").
- After waiting for a the randomized time delay, there's a visual que.
- A large black rectangle drawn as the visual trigger.
- Round results and final winner announcement with scores.

1.3. What classes did you use?

My classes:

- Main: contains the entry point, random seed generation, and game startup.
- Game: manages the game loop, rounds, scoring, and screen output.
- Random: implements a pseudo-random number generator and range function.

Built-in classes:

- Keyboard: for reading key presses.

- Output – for print the text.
- Screen – for drawing/clearing the visual trigger.
- Sys – for Sys.wait delays.
- Math – for multiplication in the random generator.
- Memory – for object allocation/deallocation.

2. How did you break down the assignment into parts?

2.1. What main parts/stages did you divide the problem into?

We divided the problem into four main parts:

- **Part 1: Random Seeding** – We created a loop that will increase a counter by incrementing as the program waits for the first key being pressed. This value seeds the random class so every session feels different as it has different wait times.
- **Part 2: The Game Loop** – We Structure a loop that runs for 5 rounds, handling the transition from “Waiting” state to “Reaction” state to “Result” state.
- **Part 3: Input Polling** – We Implemented a very fast speed polling loop that repeatedly checks the Keyboard memory until it detects either 'A' or 'B' accurately recording the first press.
- **Part 4: Score & Cleanup** – Update scores after each round, compare totals after 5 rounds display the final result and deallocate the Game object.

2.2. What functions/methods did you create?

In Main class:

- main – this our game’s entry point it generates random seed and starts the game.

In Game class:

- new – constructor; initializes scoreA and scoreB to 0.
- run – central method containing the 5-round loop and “DRAW!” logic.
- showFinalScore – compares scores and prints the final winner.
- dispose – deallocates the Game object memory.

In Random class:

- randRange(int range) – returns a random number in the desired range using a mathematical formula.

2.3. How do the parts connect to each other?

The main starts by first calling the random.setseed with a user dependent value. then it constructs a game object and calls the game.run() function.

Random.randRange is called repeatedly by the game.run method to determine a random delay, then it calls Sys.wait for that duration draws the "DRAW!" signal using Screen and Output then it polls Keyboard for the first key press, updates the appropriate score variable, and displays the result of that round. After five rounds, it calls showFinalScore. Finally, Main calls game.dispose() to free memory.

3. How does it work? (Core logic)

3.1. What algorithm did you use?

We implemented a Linear Congruential Generator (LCG) for randomness, using the formula:

```
seed = (seed * 25173 + 13849) & 32767
```

The random delay is then computed as value between 1 and 4 seconds This ensures the appearance of the "DRAW!" signal is unpredictable, as the Hack computer has no real-time clock. We chose this approach to make the game more unpredictable and make it more fun.

3.2. What variables/data structures did you use?

In Game class (fields):

- int scoreA – tracks rounds won by Player A.
- int scoreB – tracks rounds won by Player B.

In Random class (static field):

- static int seed – persists across calls to generate the sequence.

Local variables (various methods):

- int key – temporarily holds the ASCII value of the pressed key.
- int i – loop counter for rounds.

3.3. Are there loops/conditions? How do they work?

- Main round loop: while the value of i is less than 6 in Game.run The program executes rounds and increments the value of i this ensures the game is run exactly 5 rounds.

```
while (round < 6) {  
    do Screen.clearScreen();  
    do Output.moveCursor(1, 27);
```

```
do Output.println("ROUND ");

do Output.println(round);


do Output.moveCursor(10, 20);

do Output.println("Wait for it...");


// Ensure players aren't holding keys down early
while (~(Keyboard.keyPressed() = 0)) {}


// Random delay between ~1 and 4 seconds
let delay = 1000 + Random.randrange(3000);

do Sys.wait(delay);


// The Cue

do Screen.drawRect(100, 60, 412, 180);

do Output.moveCursor(11, 29);

do Output.println("DRAW!");


let roundOver = false;

while (~roundOver) {

    let key = Keyboard.keyPressed();

    if (key = 65) { // 'A' key

        let scoreA = scoreA + 1;

        do Output.moveCursor(15, 22);
```

```

        do Output.println("Player A wins round!");

        let roundOver = true;

    }

    if (key = 66) { // 'B' key

        let scoreB = scoreB + 1;

        do Output.moveCursor(15, 22);

        do Output.println("Player B wins round!");

        let roundOver = true;

    }

}

```

```

    let round = round + 1;

    do Sys.wait(1500);

}

```

```

do showFinalScore();

return;

}

```

- Key buffer clear loop: while (Keyboard.keyPressed() > 0) the program waits until both players release their keys before starting the next random delay (prevents cheating).
- Reaction polling loop: continuously reads Keyboard.keyPressed() until a valid key ('A' or 'B') is detected.
- Condition example: if (key = 65) – awards point to Player A; symmetric check for key = 66.

4. What difficulties did you have?

4.1. What problems did you encounter?

We had 3 majors difficulties:

Problem 1:

The game was predictable, the first round always had the same delay when resetting the emulator.

Problem 2:

Players could “cheat” by holding their key down and winning instantly when the signal appeared.

Problem 3:

Memory management, forgetting to dispose the Game object could cause crashes after many plays.

4.2. How did you overcome them?

Solution 1:

We Added a seed-generation loop in Main that increments while waiting for the title-screen keypress this made sure the delay depend on user timing and was random,

Solution 2:

We implemented a buffer-clear loop that waits until the Keyboard.keyPressed() returns 0 before starting the random timer This ensured the players could not cheat by holding their key down.

Solution 3:

Added a dispose() method to Game and called do game.dispose() in Main after the game ends to dealocate memory for game objects.

4.3. What new things did you learn?

New Concept 1:

Randomness in computers is often deterministic and based on mathematical formulas (LCG).

New Concept 2:

Real time interaction on systems without interrupts is achieved through really fast polling.

New Concept 3:

Proper memory management (dispose) and careful screen buffer handling are very important for making programs that don't crash and for good user experience.